

1. Introduction

This application was developed to provide fans and journalists with a simple and centralized way to access detailed information about anime.

The system is composed of several coordinated components:

- **Frontend** built with *Handlebars* and *JavaScript*, responsible for rendering the user interface and interacting with the backend.
- **Main server** implemented in *Express.js*, acting as an API gateway that routes requests to the appropriate microservices.
- **Two microservices**, each dedicated to a specific type of data:
 - **Spring Boot + PostgreSQL**, used to manage and serve static anime data such as titles, genres, studios, and staff information.
 - **Express + MongoDB**, responsible for dynamic data such as user reviews and other frequently updated information.

2.1 Main Server (Express Gateway)

Solution

The main server was implemented using **Node.js** and **Express**, and it acts as the **API Gateway** for the entire system. Its primary responsibility is to serve as the **single entry point** for the frontend, forwarding requests to the appropriate microservices:

- the **Java Spring Boot** microservice (anime data from PostgreSQL)
- the **MongoDB** microservice (user reviews)

Each endpoint receives parameters from the frontend, validates them, constructs the appropriate query, and forwards the request using **Axios**.

Issues

The main challenge was ensuring that the gateway remained fast and did not accidentally become a bottleneck. Since it must forward every request to downstream services, any delay or failure in those services directly affects the gateway's responsiveness. Another issue involved handling heterogeneous data formats coming from SQL and NoSQL microservices,

Requirements

The gateway was designed to satisfy the project requirement that the **central server must remain lightweight and must not perform heavy duties**. It also fulfills the requirement of providing a **single entry point** for the Handlebars frontend, which interacts exclusively with this server. The

architecture ensures that all database access is delegated to the specialized microservices, respecting the separation of concerns

Limitations

Because the gateway intentionally avoids implementing complex logic, it depends heavily on the performance and correctness of the downstream microservices. It does not include advanced resilience patterns such as retries, circuit breakers, meaning that slow or failing microservices will directly impact user experience.

2.2 Spring Boot Microservice (Anime + PostgreSQL)

Solution

The Java microservice was implemented using **Spring Boot** and follows the **MVC architectural pattern** to ensure a clean separation between data, business logic, and presentation layers.

Persistence is handled through **JPA (Java Persistence API)**, which provides an abstraction over SQL and allows the application to interact with the PostgreSQL database using entity classes instead of raw queries.

Each database table is mapped to a corresponding **Entity** class, and the interaction with the database is performed through **Repository interfaces**, which represent the data-access layer.

These repositories expose methods for querying the database, either through:

- automatically generated JPA methods
- custom JPQL queries
- dynamic queries built in the service layer

Issues

During development, the main challenge involved **schema mismatches between the CSV-imported database and the JPA entity definitions**.

Requirements

The microservice fulfills the project requirement of providing a **robust, strongly typed SQL backend** capable of handling complex queries and structured datasets. It ensures:

- strict schema enforcement
- reliable persistence for large relational datasets
- clean REST endpoints for the gateway
- separation of concerns between SQL logic and other microservices

Limitations

The service relies heavily on fixed values, which means any change in the database values requires updating the definitions.

The current design assumes the database schema remains stable; schema changes require updating multiple layers (entity, repository, service).

2.3 MongoDB Microservice

Solution

The MongoDB microservice was implemented as a **pure Express REST API** dedicated to handling all *dynamic* and *user-generated* data, such as reviews, comments, and ratings. It uses **Mongoose** to define flexible schemas that can evolve independently from the rigid SQL structure used by the Spring Boot service.

The microservice exposes lightweight JSON endpoints (e.g., `/reviews/:animeId`, `/reviews`) that the Express Gateway consumes via Axios. Its architecture follows a simplified MVC pattern: route modules handle HTTP requests, controllers manage logic, and Mongoose models interact with the database.

Issues

The main challenge was ensuring **data consistency** between the SQL microservice and the MongoDB microservice. Since reviews reference anime IDs stored in PostgreSQL, the system must trust that the gateway sends valid IDs.

Requirements

This microservice fulfills the requirement of managing **dynamic, user-generated content** separately from the static SQL dataset. It ensures:

- fast write operations
- flexible document structures
- independent scaling from the SQL server
- clean REST endpoints for the gateway

Limitations

Because the microservice relies on MongoDB's flexible schema, it does not enforce relational integrity. It cannot guarantee that an `animeId` stored in a review actually exists in the SQL database.

The service also lacks advanced features such as caching, rate limiting, or aggregation pipelines for analytics. As a result, heavy read operations (e.g., fetching thousands of reviews) may require future optimization.

2.4 Frontend (Handlebars + JS)

Solution

The frontend was implemented as a **single-page interface** built using **HTML, CSS, Bootstrap, Handlebars.js, and vanilla JavaScript**. Its primary role is to render anime data received from the Express Gateway and present it in a clean, responsive grid layout.

The UI uses **Handlebars partials** to generate reusable components such as anime cards, navigation elements, and search bars. These partials are compiled and injected into the page at runtime, allowing the frontend to dynamically update content without reloading the page.

JavaScript handles all asynchronous communication with the gateway Axios, populating the DOM with rendered templates. The design ensures a clear separation between **data retrieval**, **template rendering**, and **UI interaction**, making the frontend modular and easy to extend.

Issues

The main challenge involved configuring **client-side Handlebars** correctly. As seen during development, missing scripts caused errors.

Another issue was ensuring that the dynamic grid layout remained visually consistent despite receiving heterogeneous data from the SQL and MongoDB microservices. Some anime entries lacked fields such as episode count or status, requiring defensive checks in the templates.

Requirements

The frontend satisfies the project requirement of being a **One-Page Site** built with **Handlebars components**, including:

- a reusable **search bar**
- a responsive **anime catalog grid**
- modular **partials** for cards, navigation, and layout
- dynamic rendering based on API responses

Limitations

The catalog is filled dynamically, but the anime cards aren't. So any modification in the API would require a modification in the front-end too.

Loading detailed information on click may cause noticeable delays for users with slow connections.

Some anime lacked certain fields (e.g., missing score), which caused empty fields.

Conclusions

The project demonstrated the value of a modular architecture where each component has a clear responsibility. Separating SQL, NoSQL, the gateway, and the frontend simplified development and debugging. We also learned the importance of consistent data schemas, lightweight gateway design, and careful handling of dynamic rendering on the client side. Overall, the process reinforced good engineering practices such as isolation, scalability, and incremental testing.

Extra Information

To create PostgreSQL db tables run script inside:

```
""\Dylan-Timo-IUMTWEB\HCI-assignment\solution\script\main.py""
```

Required the clean data.

Mind checklist in [README.md](#) inside /script/

Bibliography

- Spring Framework Documentation, Spring.io, 2024
- MongoDB Manual, MongoDB.com, 2024
- Express.js Guide, expressjs.com, 2024
- Handlebars.js Documentation, handlebarsjs.com, 2024
- PostgreSQL Documentation, postgresql.org, 2024
- Copilot